

המדריך המלא והסופי: יסודות מדעי המחשב

ב-C#

כל חומר הלימוד המהיב לבחינת הבגרות בשאלון 899371 (כולל תוספות ודגשים)

קובץ מורחב וסופי זה כולל את כל חומר היסוד הקודם, ובנוסף שולבו בתוכו תתי-הנושאים החשובים: מניפולציה ופעולות על מחרוזות, פירוק מספרים לספרות, ומנגנון העברת פרמטרים לפי ערך (By Value).

1. טיפוסים נתונים ומשתנים קבועים

משתנה הוא תא בזיכרון המחשב המכיל ערך. לכל משתנה יש טיפוס המגדיר איזה סוג מידע הוא יכול להכיל:

שם הטיפוס ב-C#	מה הוא מייצג?	דוגמה להצהרה ואתחול בקוד
int	מספר שלם (חיובי, שלילי או אפס)	<code>int x = 5;</code>
double	מספר ממשי (שבר עשרוני)	<code>double num = 5.75;</code>
char	תו בודד (מוקף בגרש בודד)	<code>char ch = 'A';</code>
bool	ערך לוגי: אמת (true) או שקר (false)	<code>bool flag = true;</code>
string	מחרוזת טקסט (מוקפת במרכאות כפולות)	<code>string s = "Hello";</code>

קלט ופלט ב-C#:

- הדפסה למסך (פלט):** מתבצעת באמצעות `Console.WriteLine()` (יורד שורה לאחר ההדפסה) או `Console.Write()` (לא יורד שורה).
- קליטת נתונים (קלט):** הפקודה `Console.ReadLine()` קולטת תמיד ערך מסוג `string`. כדי לקלוט מספרים, יש לבצע המרה (Casting / Parsing):

```
int age = int.Parse(Console.ReadLine());
double price = double.Parse(Console.ReadLine());
```

חוקי חילוק ושארית (Modulo):

- חילוק שלמים:** חילוק `int` ב-`int` מחזיר מספר שלם בלבד (ללא השבר). לדוגמה: `5 / 2` יחזיר **2**.
- פעולת השארית (%):** מחזירה את השארית שנותרה מחילוק שלמים. לדוגמה: `5 % 2` שווה ל-**1**, ו-`10 % 14` שווה ל-**4**.

טריק קבוע בבגרות - פירוק מספרים לספרות:

כאשר רוצים לקבל מספר ולעבור על הספרות שלו אחת אחת (בדרך כלל בתוך לול while כל עוד המספר גדול מ-0):

- $num \% 10$ – שולף תמיד את **ספרת האחדות** (הספרה הימנית ביותר).

- $num / 10$ – **מוריד/מוחק** את ספרת האחדות ומקצר את המספר.

```
// דוגמה: הדפסת כל הספרות של מספר מהסוף להתחלה
int num = 145;
while (num > 0)
{
    int digit = num % 10; // בשלישי 1, בשני 4, בשלישי 5
    Console.WriteLine(digit);
    num = num / 10; // ואז ל-14, ואז ל-1, ואז ל-0 והלולה
    מסתיימת
}
```

2. עבודה עם מחרוזות (Strings)

מחרוזת היא רצף של תווים. בבגרות נדרש לדעת כיצד לחקור ולשנות מחרוזות באמצעות הפעולות הבאות:

- `s.Length` – מאפיין המחזיר את מספר התווים במחרוזת (מחזיר מספר שלם).
- **גישה לפי אינדקס** `s[i]` – ניתן לגשת לכל תו בתוך המחרוזת לפי מיקומו, החל מאינדקס 0 ועד `s.Length - 1`. הערה: התו שיוצא הוא מטיפוס `char`.
- **שרשור (חיבור) מחרוזות**: ניתן לחבר מחרוזות או להדביק להן תווים ומספרים בקלות בעזרת אופרטור ה-`+`.

```
// דוגמה: ספירה כמה פעמים האות 'A' מופיעה במחרוזת
string text = "ABRACADABRA";
int count = 0;
for (int i = 0; i < text.Length; i++)
{
    if (text[i] == 'A')
    {
        count++;
    }
}
Console.WriteLine("The letter A appears " + count + " times.");
```

3. ביטויים לוגיים ומבני בקרה (תנאים)

מבני בקרה מאפשרים לנו לפצל את מסלול ריצת התוכנית בהתאם לתנאים לוגיים.

אופרטורים לוגיים:

- `&&` (וגם - AND): הביטוי כולו אמת רק אם שני התנאים מתקיימים.
- `||` (או - OR): הביטוי כולו אמת אם לפחות אחד מהתנאים מתקיים.
- `!` (לא - NOT): הופך את התוצאה הלוגית.

מבנה משפט התנאי ב-C#:

```
if (grade >= 55)
{
    Console.WriteLine("Passed");
}
else
{
    Console.WriteLine("Failed");
}
```

4. לולאות (מבני חזרה)

הלולאות מאפשרות לבצע קטע קוד שוב ושוב.

לולאת `while` (לולאה מותנית):

רצה כל עוד התנאי שבסוגריים מתקיים. משמשת כשמספר החזרות אינו ידוע מראש.

```
int count = 1;
while (count <= 5)
{
    Console.WriteLine(count);
    count++;
}
```

לולאת `for` (לולאה ספורה):

משמשת כאשר אנו יודעים בדיוק כמה פעמים אנו רוצים שהלולאה תרוץ.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine("Number: " + i);
}
```

5. מערכים חד-ממדיים ב-C#

מערך הוא אוסף תאים רציף בזיכרון מאותו הטיפוס. הגישה לכל תא מתבצעת באמצעות אינדקס המתחיל מ-0.

```
int[] grades = new int[5]; // הגדרה בגודל 5
```

מאפיין האורך וסריקה:

ב-C# משתמשים במאפיין `Length` (עם `L` גדולה) כדי לקבל את גודל המערך.

```
// קליטת ערכים לתוך מערך וסכימה שלהם
int sum = 0;
for (int i = 0; i < grades.Length; i++)
{
    grades[i] = int.Parse(Console.ReadLine());
    sum += grades[i];
}
```

6. תת-תוכניות (פעולות / פונקציות) והעברת פרמטרים

פעולה היא קטע קוד עצמאי שמבצע משימה מוגדרת. בשאלון 899371, כל הפעולות שתידרשו לכתוב יהיו מסומנות כ-`public static`.

דגש תיאורטי חשוב: העברת פרמטרים "לפי ערך" (By Value)

כשמעבירים משתנה מטיפוס פשוט (כמו `bool`, `char`, `double`, `int`) לתוך פעולה, הפעולה מקבלת העתק של הערך בלבד. כל שינוי שנעשה במשתנה בתוך הפעולה לא ישפיע על המשתנה המקורי שנמצא בתוכנית הראשית (`Main`).

דוגמה לפעולה שמחזירה ערך:

```
public static int GetMax(int num1, int num2)
{
    if (num1 > num2)
    {
        return num1;
    }
    return num2;
}
```

7. דגשים קריטיים לפתרון שאלות מעקב ומבחן

- טבלת מעקב:** פתחו עמודה לכל משתנה, כולל משתני לולאה (`i`). פתחו עמודה נפרדת עבור תנאים לוגיים (רשמו בה `T` או `F`) ועמודה לפלט המודפס למסך.
- חריגה מגבולות מערך או מחרוזת:** ודאו שהלולאות שלכם תמיד נעצרות לפני האורך של המערך/המחרוזת (`i < arr.Length`). גישה לאינדקס ששווה לאורך תגרום לשגיאת קריסה!
- רגישות לאותיות גדולות (Case-Sensitivity):** הקפידו לכתוב `Console.WriteLine` עם `C` ו-`W` גדולות, `int.Parse` עם `P` גדולה, ומשתנים בדיוק כפי שהוגדרו.